Exports
Spec: https://www.haskell.org/onlinereport/haskell2010/haskellch5.html#x11-980005

e.g. basic usage

```
module MyModule (
 myFn,
 MyData(..),
) where

import Data.List

myFn x = x + 1
data MyData = A | B
```

```
## Basic
module MyModule (
 x,                          -- export variable
 fn,
 (+),                        -- export operator
 type (+),                   --  export type operator (with GHC
extension TypeOperators)
 module Data.List
) where


import Data.List             -- imports MUST be after where
x = 1                        -- definitions
fn t = t + 1
```

```
import Data.List                -- ERROR. Imports MUST be after
where
module MyModule ( ... ) where
 ...
```

```
-- data types, classes
module MyModule (
 -- data type
 MyDataType,                                  -- Type only
 MyDataType(MyDataA, MyDataB),                -- Type + constructors
 MyDataType(v1, v2),                          -- Type + fields
 MyDataType(MyDataA, MyDataB, v1, v2),        -- Type + constructors
+ fields
 MyDataType(...),                               -- All

 -- type synonym
 MyType,

 -- newtype
```

```
 MyNewType,             -- Type only
 MyNewType(...),          -- all

 MyClass,                 -- class only
 MyClass(myfn1, myfn2),  -- class + methods
 MyClass(...),              -- all

 -- pattern
 pattern MyPattern
) where

data MyDataType a b = MyDataA a | MyDataB b
type MyType = Int
newtype MyNewType a = MkNewType a

class MyClass a where
 myfn1 :: ...
 myfn2 :: ...
```

Imports
Spec: https://www.haskell.org/onlinereport/haskell2010/haskellch5.html#x11-980005

```
-- Basic
import Data.Maybe
isJust (Just 1)


-- qualified names: i.e. full names
Prelude.id                -- variable
(Prelude.+)               -- operator
Prelude.Int               -- type
Prelude.True              -- constructor
Prelude.Num               -- type class
Prelude.return            -- class method


-- Partial imports
import Data.Maybe ()                              -- Import nothing
import Data.Maybe (fromJust, isJust)              -- Import only
fromJust,isJust
import GHC.Num ((+))                              -- Import operator
import GPC.TypeLits (type (+))                    -- Type
operator (with GHC extension TypeOperators)
import Data.Maybe (Maybe)                         -- Import type only
import Data.Maybe (Maybe(Just,Nothing))           -- type +
constructors
import Data.Maybe (Maybe(..))                     -- all
import MyLib (pattern A)                          -- patterns
import Data.Maybe hiding ()                         -- Import all
import Data.Maybe hiding (fromJust)                 -- imports all
except fromJust.
import Data.Maybe hiding (Maybe)                    -- all except
Maybe type
```

```
import Data.Maybe hiding (Maybe(Just,Nothing))   -- type +
constructors
import Data.Maybe hiding (Maybe(..))             -- all


-- Alias
import Data.List                 -- no alias
sort                             -- OK unqualified
Data.List.sort                   -- OK qualified with full name
import Data.List as D            -- alias
sort                             -- OK unqualified
D.list.sort                      -- OK qualified with alias

import Data.List (sort) as D     -- ERROR
import Data.List as d            -- ERROR


-- multiple
-- i.e. group under same alias
import Data.Char as C
import Data.Maybe as C
import Data.Either as C
c.isAlpha            -- Data.Char.isAlpha
c.fromJust           -- Data.Maybe.fromJust
c.fromLeft           -- Data.Either.fromLeft


-- Qualified only: i.e. require full name
import qualified Data.List       -- qualified only
sort                             -- ERROR unqualified
Data.List.sort                   -- OK qualified

import qualified Data.List as D  -- qualified only + alias
sort                             -- ERROR unqualified
D.sort                           -- OK qualified


-- [GHC extension, default] qualified as postfix
-- Ref: https://downloads.haskell.org/ghc/latest/docs/users_guide/
exts/import_qualified_post.html#extension-ImportQualifiedPost
import Data.Maybe qualified
import Data.Maybe qualified as D
```